🍽️ AI Restaurant Copilot

AI-powered restaurant intelligence platform that combines revenue optimization, voice-based order taking, and POS-ready order management. Built for restaurant owners to maximize profitability through data-driven insights and automated ordering systems.

▲ Deploy

✨ Core Features

🤖 AI Revenue Intelligence Engine

- **Contribution Margin Analysis** - Real-time profitability tracking per item/channel
- **Item Profitability Insights** - Margin analysis with commission impact
- **Sales Velocity Detection** - Hot/cold item identification with trend analysis
- **Hidden High-Margin Items** - Discover underperforming profitable items
- **Low-Margin Risk Alerts** - Identify items losing money online
- **AI Combo Recommendations** - Co-occurrence analysis for bundle suggestions
- **Price Optimization** - Channel-aware pricing recommendations with revenue impact

📞 AI Voice Ordering Copilot

- **Natural Language Processing** - Speech-to-text with intent recognition
- **Multi-language Support** - English + Hindi voice commands
- **Smart Order Parsing** - Item recognition with modifiers (spicy, extra, etc.)
- **Real-time Upsell Suggestions** - AI-driven cross-sell recommendations
- **Order Confirmation** - Structured order validation and finalization
- **POS-Ready Order Generation** - Direct integration with restaurant systems

🏪 Restaurant Management Suite

- **CSV/Excel Menu Import** - Bulk menu upload with cost tracking
- **Order Logs Dashboard** - Transaction history with analytics
- **Revenue Analytics Dashboard** - Real-time business metrics
- **Channel Commission Management** - Platform-specific fee configuration
- **Order Simulation** - Test AI recommendations with virtual orders

🏗️ System Architecture

Customer Journey Flow

```
📞 Customer Calls Restaurant
    ↓
🎤 Twilio Voice Processing
    ↓
🧠 AI Intent Recognition (voiceEngine.ts)
    ↓
📋 Order Understanding & Validation
    ↓
💡 AI Upsell Recommendations
    ↓
✅ Order Confirmation & Generation
    ↓
🖥️ POS-Ready Order Creation
```

Technical Architecture

```
┌─────────────────┐    ┌─────────────────┐    ┌─────────────────┐
│   React UI      │    │  Vercel API     │    │   Supabase      │
│   (Frontend)    │◄──►│  (Serverless)   │◄──►│   (Database)    │
│                 │    │                 │    │                 │
│ • Dashboard     │    │ • Order APIs    │    │ • Orders        │
│ • Voice Logs    │    │ • Menu APIs     │    │ • Call Logs     │
│ • Analytics     │    │ • Voice Webhook │    │ • Channels      │
│ • POS Simulation│    │ • AI Processing │    │ • Menu Items    │
└─────────────────┘    └─────────────────┘    └─────────────────┘
        │                      │                      │
        └──────────────────────┼──────────────────────┘
                               ▼
                    ┌─────────────────┐
                    │   Twilio Voice  │
                    │   (Telephony)   │
                    └─────────────────┘
```

🛠️ Tech Stack

Frontend

- **React 18** + **TypeScript** - Modern component architecture
- **Vite** - Fast build tool and dev server
- **Tailwind CSS** + **shadcn/ui** - Utility-first styling
- **Framer Motion** - Smooth animations and transitions

- **React Router** - Client-side routing
- **Recharts** - Data visualization components
- **Sonner** - Toast notifications

Backend

- **Vercel Serverless Functions** - Scalable API endpoints
- **Supabase** - PostgreSQL database with real-time features
- **Twilio Voice API** - Phone call processing and speech recognition
- **PapaParse** + **SheetJS** - CSV/Excel file processing

AI & Intelligence

- **Custom AI Engine** (`src/lib/aiEngine.ts`) - Revenue optimization algorithms
- **Voice Processing Engine** (`src/lib/voiceEngine.ts`) - NLP and intent recognition
- **Co-occurrence Analysis** - Combo recommendation engine
- **Sales Velocity Algorithms** - Trend detection and forecasting

Development Tools

- **ESLint** + **Prettier** - Code quality and formatting
- **Vitest** - Unit testing framework
- **TypeScript** - Type-safe development

🚀 Quick Start

Prerequisites

- **Node.js 18+**
- **npm** or **bun**
- **Supabase Account** (supabase.com)
- **Twilio Account** (twilio.com) - for voice features

1. Clone & Install

```
git clone <your-repo-url>
cd restaurant-ai-copilot
npm install
```

2. Environment Configuration

```
cp .env.example .env.local
```

Configure `.env.local` :

```
# Supabase Configuration (Required)
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# Twilio Voice Configuration (For Voice Ordering)
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_PHONE_NUMBER=+1234567890
RESTAURANT_FALLBACK_PHONE=+1234567890

# Optional: Single Restaurant Mode
RESTAURANT_ID=your-restaurant-uuid
```

3. Database Setup

1. **Create Supabase Project**
2. **Run Schema Migration**:
   - Open Supabase SQL Editor
   - Copy entire content from `supabase_schema.sql`
   - Execute the SQL

4. Development Server

```
npm run dev
```

Visit `http://localhost:5173`

5. Voice Setup (Optional)

1. **Deploy to Vercel** for public webhook URL
2. **Configure Twilio Webhook**:
   - Go to Twilio Console → Phone Numbers
   - Set Voice Webhook: `POST https://your-domain.vercel.app/api/voice`

📋 API Reference

Core Endpoints

Menu Management

- `GET|POST|PUT|DELETE /api/menu` - CRUD operations

- `POST /api/menu/upload` - Bulk CSV/Excel import
- **Rate Limit**: 1000 items per request

Order Management

- `GET /api/orders` - Fetch orders with analytics
- `POST /api/orders` - Create new order
- `POST /api/orders/create` - Create order from cart
- `POST /api/orders/upload` - Bulk order import
- **Rate Limit**: 5000 rows per request

Restaurant Configuration

- `GET|PUT /api/restaurants/profile` - Restaurant settings
- `GET|POST|DELETE /api/channels` - Commission management

Voice & Telephony

- `POST /api/voice` - Twilio voice webhook entry point
- `POST /api/process-order` - Voice order processing
- `GET /api/calls/recent` - Call logs dashboard

Data Formats

Menu CSV Format

```
Item Name,Selling Price,Food Cost,Category
Margherita Pizza,299,120,Pizza
Chicken Burger,199,80,Burgers
```

Order CSV Format

```
Order ID,Item Name,Quantity,Channel,Timestamp
ORD-001,Margherita Pizza,2,ZOMATO,2024-01-15 14:30:00
ORD-002,Chicken Burger,1,OFFLINE,2024-01-15 15:45:00
```

🗂️ Project Structure

```
restaurant-ai-copilot/
├── api/                          # Vercel serverless functions
│   ├── menu/                     # Menu CRUD operations
│   ├── orders/                   # Order management & analytics
│   ├── voice.ts                  # Twilio voice webhook
│   ├── process-order.ts          # Voice order processing
│   ├── calls/                    # Call logs & analytics
│   └── _lib/                     # Shared utilities
│       ├── auth.ts              # Supabase authentication
│       ├── twilioVoice.ts       # Voice processing helpers
│       └── callSessionStore.ts  # Voice session management
├── src/
│   ├── components/               # Reusable React components
│   │   ├── ui/                  # shadcn/ui components
│   │   └── landing/             # Landing page components
│   ├── pages/                   # Main application pages
│   │   ├── Dashboard.tsx        # Main analytics dashboard
│   │   ├── VoiceCopilot.tsx     # Call logs & voice analytics
│   │   ├── Orders.tsx           # Order transaction logs
│   │   ├── MenuIntelligence.tsx # AI menu insights
│   │   └── OrdersSimulation.tsx # POS simulation
│   ├── lib/                     # Core business logic
│   │   ├── aiEngine.ts          # Revenue intelligence algorithms
│   │   ├── voiceEngine.ts       # Voice processing & NLP
│   │   ├── restaurantData.tsx   # Data context & API client
│   │   ├── csvParser.ts         # File import utilities
│   │   └── types.ts             # TypeScript definitions
│   └── assets/                  # Static assets
├── supabase_schema.sql          # Database schema
├── vercel.json                  # Vercel deployment config
└── package.json                 # Dependencies & scripts
```

🔐 Security & Authentication

Authentication Flow

- **Supabase Auth** - JWT-based user authentication
- **Restaurant-scoped Access** - Users only see their restaurant data
- **API Key Protection** - Sensitive keys stored as environment variables

Data Protection

- **Row Level Security** - Supabase RLS policies

- **Input Validation** - Server-side validation on all endpoints
- **SQL Injection Prevention** - Parameterized queries

Voice Security

- **Call SID Validation** - Twilio webhook verification
- **Session Management** - Secure voice session handling
- **Transcript Privacy** - Encrypted call data storage

📊 Database Schema

Core Tables

```
-- User authentication (via Supabase Auth)
auth.users

-- Restaurant profiles
restaurants (
  id, user_id, name, location, cuisine,
  uses_pos, setup_complete, pos_config
)

-- Menu items with pricing
menu_items (
  id, restaurant_id, item_name, selling_price,
  food_cost, category, aliases
)

-- Order transactions (item-level)
orders (
  id, restaurant_id, order_id, order_number,
  item_name, quantity, channel, timestamp,
  delivery_address, city, pincode,
  food_total, delivery_charge, total_amount
)

-- Sales channels & commissions
channels (
  id, restaurant_id, name, commission_percentage, enabled
)

-- Voice call logs
call_logs (
  id, call_sid, restaurant_id, status,
  transcript, order_json, total
)
```

Key Relationships

- **1 Restaurant → N Menu Items**
- **1 Restaurant → N Orders**
- **1 Restaurant → N Channels**
- **1 Restaurant → N Call Logs**

🎯 AI Algorithms Overview

Revenue Intelligence Engine

- **Margin Calculation**: `(selling_price - food_cost) / selling_price * 100`
- **Online Margin**: Accounts for platform commissions
- **Sales Velocity**: Trend analysis using moving averages
- **Co-occurrence Analysis**: Identifies frequently ordered item combinations

Voice Processing Engine

- **Intent Recognition**: Regex-based pattern matching for order intents
- **Entity Extraction**: Menu item identification with fuzzy matching
- **Modifier Parsing**: Size, spice level, add-on recognition
- **Conversation State**: Context-aware dialogue management

Recommendation Systems

- **Price Optimization**: Channel-aware pricing with revenue impact prediction
- **Combo Suggestions**: Statistical co-occurrence analysis
- **Upsell Recommendations**: Cross-sell based on order patterns

🚀 Deployment

Vercel Deployment (Recommended)

```
npm i -g vercel
vercel login
vercel --prod
```

Required Environment Variables in Vercel:

- All Supabase configuration
- Twilio credentials (for voice features)
- `RESTAURANT_ID` (optional, for single-restaurant mode)

Manual Deployment

1. **Build**: `npm run build`
2. **Deploy** `dist/` to your hosting provider
3. **Configure** API routes to `/api/*`

🧪 Demo Limitations

Current Hackathon Demo Restrictions:

🔒 Single Restaurant Mode

- **Fixed Restaurant**: Currently configured for "Darpan's Restro"
- **Email Restriction**: Only `darpanparmar1707@gmail.com` can access voice logs
- **Restaurant ID**: Hardcoded for demo purposes

📞 Voice Ordering Limits

- **API Rate Limits**: Twilio usage restrictions for demo
- **Regional Focus**: Optimized for Indian English + Hindi
- **Menu Scope**: Limited to demo menu items

🔧 Technical Constraints

- **Database**: Shared demo database with sample data
- **POS Integration**: Mock integration for demonstration
- **Analytics**: Limited historical data scope

🌟 Production Readiness

The architecture already supports:

✅ Multi-Restaurant Scaling

- Dynamic restaurant onboarding
- Isolated data per restaurant
- Configurable commission rates

✅ Real POS Integrations

- RESTful API endpoints
- Order format standardization
- Real-time sync capabilities

✅ Advanced Voice Features

- Multi-language support expansion
- Regional accent optimization
- Enhanced NLP models

✅ Enterprise Features

- User role management
- Advanced analytics
- API rate limiting
- Audit logging

🛠️ Development Commands

```
# Development
npm run dev              # Start dev server
npm run build           # Production build
npm run preview         # Preview build

# Code Quality
npm run lint            # ESLint check
npm run test            # Run tests
npm run test:watch      # Watch mode tests

# Deployment
npm run build:dev       # Development build
```

📈 Performance Optimizations

Frontend

- **Lazy Loading**: Route-based code splitting
- **Memoization**: Expensive calculations cached
- **Virtual Scrolling**: Large lists optimized

Backend

- **Database Indexing**: Optimized queries
- **Connection Pooling**: Supabase connection management
- **Caching**: API response caching

AI Processing

- **Batch Processing**: Bulk analytics calculations
- **Incremental Updates**: Real-time data processing
- **Memory Optimization**: Efficient algorithm implementations

🤝 Contributing

1. **Fork** the repository
2. **Create** feature branch: `git checkout -b feature-name`
3. **Commit** changes: `git commit -m 'Add feature'`
4. **Push** branch: `git push origin feature-name`
5. **Open** Pull Request

Development Guidelines

- **TypeScript Strict**: All code must pass type checking

- **ESLint Compliance**: Follow coding standards
- **Test Coverage**: Unit tests for critical functions
- **Documentation**: Update README for new features

📄 License

MIT License - See LICENSE file for details.

🆘 Support & Documentation

- 📧 **Email**: your-email@example.com
- 🐛 **Issues**: GitHub Issues
- 💬 **Discussions**: GitHub Discussions
- 📖 **Wiki**: Project Wiki

Built with ❤️ for restaurant owners to maximize profitability through AI-driven insights and automation.

- **Prices**: Must be positive numbers
- **Quantities**: Must be integers ≥ 1

File Upload Limits

- **CSV/Excel**: Max 10MB
- **Images**: Not supported (text-only)

🛠️ Development

Project Structure

```
├── api/                    # Vercel serverless functions
│   ├── menu/              # Menu CRUD
│   ├── orders/            # Order management
│   ├── voice/             # Twilio webhooks
│   └── _lib/              # Shared utilities
├── src/
│   ├── components/        # React components
│   ├── pages/             # Page components
│   ├── lib/               # Utilities & services
│   └── assets/            # Static assets
├── supabase_schema.sql    # Database schema
└── package.json
```

Key Files

- `src/lib/restaurantData.tsx` - Data context & API calls
- `src/lib/aiEngine.ts` - AI analytics engine
- `src/lib/csvParser.ts` - File import logic
- `api/_lib/auth.ts` - Authentication helpers

Scripts

```
npm run dev          # Development server
npm run build        # Production build
npm run preview      # Preview build
npm run test         # Run tests
npm run lint         # ESLint check
```

🚀 Deployment

Vercel (Recommended)

```
npm i -g vercel
vercel login
vercel --prod
```

Required Environment Variables in Vercel:

- All Supabase variables
- Twilio variables (if using voice)
- `CALL_AGENT_OWNER_EMAIL` (optional)

Manual Deployment

1. Build: `npm run build`
2. Deploy `dist/` to your hosting
3. Configure API routes to `/api/*`

🐛 Troubleshooting

Common Issues

Orders not showing in logs:

- Check database schema is updated
- Verify `restaurant_id` matches your user
- Check browser console for errors

Voice calls not working:

- Verify Twilio webhook URL is correct
- Check Twilio credentials
- Ensure phone number is configured

CSV import fails:

- Check column headers match expected format
- Verify data types (numbers, dates)
- Check file size < 10MB

Authentication errors:

- Verify Supabase keys are correct
- Check user is logged in
- Ensure restaurant profile exists

Debug Mode

```
# Enable verbose logging
DEBUG=* npm run dev
```

📈 Performance

Optimizations

- **Database Indexing** on frequently queried columns
- **Pagination** for large datasets (12 items/page)
- **Lazy Loading** for analytics calculations
- **Memoization** for expensive computations

Monitoring

- **Vercel Analytics** for API performance
- **Supabase Dashboard** for database metrics
- **Browser DevTools** for frontend performance

🤝 Contributing

1. Fork the repository
2. Create feature branch: `git checkout -b feature-name`
3. Commit changes: `git commit -m 'Add feature'`
4. Push to branch: `git push origin feature-name`
5. Open Pull Request

Code Style

- **ESLint** + **Prettier** configured
- **TypeScript** strict mode
- **Conventional commits**

📄 License

MIT License - see LICENSE file.

🆘 Support

- **Issues**: GitHub Issues
- **Discussions**: GitHub Discussions
- **Email**: your-email@example.com

Built with ❤️ for restaurant owners